INFORME

Aplicación de los Patrones MVC y Strategy – Sistema de Gestión de Clínica Odontológica

Proceso de Desarrollo de Software – UADE 2026

Integrantes: Daher, Ezequiel · Quesada, Bautista · Claros, Alex

1. Introducción

MVC (Modelo-Vista-Controlador) es un patrón que separa una aplicación en tres partes: el modelo, que tiene los datos y las reglas del negocio; la vista, que es lo que ve el usuario; y el controlador, que recibe las acciones de la vista y se las pasa al modelo. Así se puede cambiar una pantalla sin tocar las reglas, y al revés.

Al revisar el proyecto vimos que la estructura ya era MVC: hay un paquete para la vista, uno para los controladores y el resto forma el modelo. El problema era que la separación funcionaba solo en un sentido. La vista le mandaba datos al controlador con DTOs, pero recibía de vuelta las entidades del modelo, y por eso casi todas las clases de la vista conocían el modelo. Además, los DTOs llevaban objetos del modelo, las opciones de los combos no pasaban por el controlador y los controladores no validaban nada.

Corregimos los cuatro problemas (P-01 a P-04) sin rehacer el sistema, y los detallamos más abajo. Una solución que ya funcionaba en un controlador, buscarDatosPacientePorId, que devolvía un DTO en lugar de la entidad, se extendió a todos. También revisamos el patrón Strategy, que el sistema ya aplica en el cálculo del monto según la cobertura del paciente, y lo documentamos en la sección 5.

2. Metodología de Trabajo

Cada hallazgo se trató en tres pasos.

1. **Revisión por capa.** Se verificó qué paquetes importaba cada clase y qué tipos recibían y devolvían los métodos públicos de los controladores.
2. **Contraste con el patrón.** Cada situación se comparó con lo que MVC asigna a cada componente. El modelo representa los datos y aplica las reglas del negocio. El controlador responde a las acciones del usuario, invoca al modelo y es el intermediario con la vista. La vista presenta los datos y se comunica con el controlador mediante DTOs. Para la validación se tomó el criterio de tres niveles: la vista controla formato y campos requeridos, el controlador vuelve a validar consistencia y campos requeridos, y el modelo aplica las validaciones del negocio.

3. **Elección de la corrección más simple.** Se evitó agregar clases que no respondieran a una necesidad concreta.

3. Resumen de Hallazgos

Los cuatro problemas afectaban la comunicación entre la vista y el resto del sistema; ninguno estaba dentro del modelo.

ID	Problema detectado	Regla MVC aplicada	Dónde verlo
P-01	La vista trabajaba con las entidades del modelo	Vista y controlador se comunican mediante DTOs	Controladores, DTOs de salida y paneles de la vista
P-02	Los DTOs de entrada llevaban objetos del modelo	Vista y controlador se comunican mediante DTOs	PacienteRegistro, OdontologoRegistro y TurnoEdicion
P-03	Las opciones de los combos no pasaban por el controlador	El modelo envía datos a la vista a través del controlador	Main, MainFrame y métodos listar de los controladores
P-04	El controlador no validaba los datos recibidos	Validación en tres niveles: vista, controlador y modelo	Controladores, servicios y TurnoValidador

4. Correcciones Implementadas

P-01 – La vista trabajaba con las entidades del modelo

Situación previa. Los métodos de consulta y de alta de los cuatro controladores devolvían entidades: registrarPaciente devolvía un Paciente, listarOdontologos una lista de Odontologo y listarTurnos una lista de Turno. Las tablas, formularios y combos leían los datos directamente de esos objetos, por ejemplo con turno.getNombrePaciente() u odontologo.getTarifaBase(). Así, quince de los dieciocho archivos del paquete view importaban clases de entity. La vista tenía además a su alcance métodos que modifican el modelo, como Turno.cambiarEstado, Turno.actualizar o Paciente.actualizarDatos, sin pasar por un controlador. OpcionEntidad, el ítem de los combos de turnos, guardaba la entidad completa solo para obtener su ID.

Regla MVC aplicada. La vista presenta los datos y envía las acciones del usuario al controlador, y la comunicación entre ambos debe hacerse con DTOs. El controlador recibe del modelo los datos solicitados y los entrega a la vista sin exponer el modelo. Si la vista

conoce las entidades, un cambio en el modelo obliga a modificar la vista.

Cambio aplicado. Se crearon cuatro DTOs de salida: PacienteDatos, OdontologoDatos, SecretariaDatos y TurnoDatos. Cada uno extiende al DTO de registro de su actor y le suma el ID y los datos que solo se muestran, como la tarifa del odontólogo o los nombres de los actores de un turno. Cada controlador tiene un método privado aDatos que traduce la entidad al DTO, y todos sus métodos públicos devuelven DTOs. Como cada DTO de salida es también un DTO de registro, los formularios conservaron su método cargar. OpcionEntidad pasó a guardar solo el ID y la etiqueta. Se eliminó PacienteService.buscarDatosPorId, que armaba el DTO dentro del servicio, porque esa traducción ahora es tarea del controlador.

Dónde verlo. Los cuatro DTOs nuevos del paquete dto, el método aDatos de cada controlador y los paneles de tabla y formulario. Una búsqueda sobre el código confirma que ningún archivo del paquete view importa clases de entity.

P-02 – Los DTOs de entrada llevaban objetos del modelo

Situación previa. PacienteRegistro guardaba la cobertura como un CoberturaPaciente, OdontologoRegistro la especialidad como un EspecialidadOdontologica y TurnoEdicion el estado como un EstadoTurno. La vista llenaba los combos con esos objetos y los devolvía dentro del DTO. CoberturaPaciente, además, contiene la regla de cálculo del monto a facturar, de modo que la vista manipulaba una pieza de la lógica del negocio. El paquete dto dependía del paquete entity.

Regla MVC aplicada. Un DTO solo transporta datos entre la vista y el controlador. Si lleva objetos del modelo, la vista sigue dependiendo de él de forma indirecta.

Cambio aplicado. Los tres campos pasaron a ser de tipo String. La vista envía la descripción elegida en el combo, por ejemplo "Obra Social" u "Ortodoncia", y el modelo la convierte con herramientas que ya existían: PacienteServiceImpl usa CatalogoCoberturas, OdontologoServiceImpl usa la búsqueda de especialidades de OdontologoFactory y TurnoServiceImpl usa el enum EstadoTurno. PacienteServiceImpl recibe ahora el catálogo de coberturas por constructor. El formato de los archivos de datos no cambió, porque la persistencia sigue guardando el código de cada cobertura y especialidad.

Dónde verlo. Los campos cobertura, especialidad y estado de los tres DTOs, el constructor de PacienteServiceImpl y los métodos registrar y actualizar de los servicios. El paquete dto ya no importa ninguna clase de entity.

P-03 – Las opciones de los combos no pasaban por el controlador

Situación previa. Main obtenía las listas de coberturas y especialidades desde DependenciasClinica, que las sacaba del catálogo de coberturas y de la fábrica de odontólogos. Esas listas se pasaban por constructor a MainFrame, que las entregaba a PacientePanel y OdontologoPanel. Los estados de un turno se tomaban dentro de la vista con EstadoTurno.values(). En los tres casos la vista recibía datos del modelo sin que interviniera un controlador.

Regla MVC aplicada. El modelo envía a la vista los datos solicitados a través del controlador, no por un camino lateral ni desde la clase que arma la aplicación.

Cambio aplicado. Se agregaron listarCoberturas en PacienteController, listarEspecialidades en OdontologoController y listarEstados en TurnoController, que devuelven las opciones como listas de texto. Cada panel pide sus opciones a su controlador al construirse. Se quitaron de DependenciasClinica los métodos getCoberturas y getEspecialidades, y MainFrame dejó de recibir esas listas.

Dónde verlo. Los métodos listar de los controladores, el constructor de MainFrame, que ahora recibe solo los cuatro controladores, y los constructores de PacientePanel, OdontologoPanel y TurnoFormularioPanel.

P-04 – El controlador no validaba los datos recibidos

Situación previa. Los controladores solo delegaban cada llamada en su servicio. Las validaciones de campos (nombre no vacío, solo letras, DNI positivo, email y domicilio completos) estaban en los servicios, mezcladas con las reglas del negocio, como el DNI duplicado o la especialidad que no puede modificarse. En turnos ocurría lo mismo: TurnoValidador controlaba que fecha, hora, motivo y estado no fueran nulos, junto con la regla de que el odontólogo pueda atender el motivo de consulta.

Regla MVC aplicada. La validación se reparte en tres niveles: la vista controla formato y campos requeridos, el controlador vuelve a validar consistencia y campos requeridos, y el modelo realiza las validaciones del negocio. En el sistema existían el primer y el tercer nivel; el segundo faltaba y su tarea la cumplía el modelo.

Cambio aplicado. Los métodos validarDatos de PacienteServiceImpl, OdontologoServiceImpl y SecretariaServiceImpl se trasladaron a sus controladores, junto con el control de que el DTO no sea nulo. TurnoController incorporó la validación de fecha, hora, motivo y estado, y TurnoValidador quedó solo con la regla de negocio, en validarOdontologoHabilitado. Los servicios conservan las reglas del negocio y las entidades conservan las invariantes que verifican en su constructor, así que el modelo sigue protegido aunque se lo use sin un controlador. Las validaciones se trasladaron sin cambios y los mensajes que ve el usuario son los mismos.

Dónde verlo. El método validarDatos de los controladores de pacientes, odontólogos y secretarias, el método validarCampos de TurnoController y TurnoValidador, que pasó de 45 a 22 líneas. Los servicios se redujeron: PacienteServiceImpl pasó de 139 a 107 líneas, OdontologoServiceImpl de 121 a 108 y SecretariaServiceImpl de 103 a 86.

5. Patrón Strategy

No se detectó un problema de diseño respecto del patrón Strategy, porque el sistema ya lo aplica en el cálculo del monto que paga cada paciente según su cobertura. Por eso no se modificó la lógica: esta sección documenta cómo la solución existente se corresponde con el patrón, y en el código solo se agregaron comentarios breves que nombran el rol de cada clase.

Strategy es un patrón de comportamiento que encapsula distintas formas de resolver un mismo problema en clases separadas, y permite cambiar en ejecución la forma en que un tercero lo resuelve. Se sugiere usarlo cuando un objeto debe realizar la misma acción con algoritmos distintos. En la clínica, la acción es calcular cuánto paga el paciente por un turno, y los algoritmos son las coberturas. Esta estructura surgió en la iteración SOLID (hallazgo P-06), cuando las coberturas pasaron de un enum a una interfaz con una clase por cobertura, para poder agregar nuevas sin editar código existente.

Componentes. CoberturaPaciente es la interfaz Strategy: define calcularMonto(odontologo), junto con getCodigo y getDescripcion. CoberturaParticular y CoberturaObraSocial son las estrategias concretas. La primera cobra la tarifa base del odontólogo, y la segunda un copago fijo de $10.000, sin importar el odontólogo. Paciente es el contexto: guarda una cobertura y le delega el cálculo. CatalogoCoberturas registra las estrategias disponibles y entrega la que corresponde según su código.

Funcionamiento. Cuando se pide el monto de un turno, la llamada baja por una cadena hasta el paciente: FacturacionTurnoService busca el turno y se lo pasa a Facturador, que llama a Turno.calcularMonto, que a su vez llama a Paciente.calcularMonto con su odontólogo. Allí el paciente usa la cobertura que tiene guardada, sin ningún condicional sobre su tipo. La cobertura puede cambiar en ejecución, porque actualizar un paciente reemplaza la estrategia que guarda.

Extensión. Agregar una cobertura nueva, por ejemplo una prepaga con descuento porcentual, requiere una clase que implemente CoberturaPaciente y su registro en CatalogoCoberturas, dentro de DependenciasClinica. Paciente, Turno y Facturador no cambian.

Dónde verlo. Los comentarios agregados en CoberturaPaciente, CoberturaParticular, CoberturaObraSocial y Paciente.calcularMonto. Como no hubo cambios de estructura, no se modificó el diagrama de clases.